# Architecture SOA et services web

## Chapitre 1: Technologies XML
## Schéma XML



# Schéma XML

- Un Schéma XML (XML schema Definition XSD) est un langage basé sur XML qui permet la définition de schémas (structure + type de données) des documents XML et facilite la communication entre applications.

- XML Schema est une alternative (successeur) pour les DTDs.

| DTD | XML schema |
|---|---|
| N'est pas une syntaxe XML | Basé sur la syntaxe XML |
| Difficile à étendre | Facilement extensible |
| Données textuelles non typées | Supporte les types de données |
| Ne permet pas de spécifier exactement le nombre d'occurrences d'un élément | Permet de spécifier exactement le nombre d'occurrences d'un élément |
| Ne supporte pas les espaces de noms | Supporte les espaces de noms |

# Schéma XML

- Un schéma d'un document définit:
  - les **éléments** possibles dans le document
  - les **attributs** associés à ces éléments
  - la **structure** du document
  - les **types** de données

- Le schéma est spécifié en XML:
  - pas de nouveau langage
  - balisage de déclaration
  - espace de nom spécifique **xsd** : ou **xs**

- Présente de nombreux avantages
  - structures de données avec types de données
  - extensibilité par **héritage**
  - analysable à partir d'un parseur XML standard

# Schéma XML: exemple

**Un simple document XML**

```
<?xml version="1.0"?>
<note>
 <to>Tove</to>
 <from>Jani</from>
 <heading>Reminder</heading>
 <body>Don't forget me this weekend!</body>
</note>
```

**Un schéma XML**

```
<?xml version="1.0"?>
<xs:schema xmlns:xs=http://www.w3.org/2001/XMLSchema
elementFormDefault="qualified">
<xs:element name="note">
 <xs:complexType>
  <xs:sequence>
   <xs:element name="to" type="xs:string"/>
   <xs:element name="from" type="xs:string"/>
   <xs:element name="heading" type="xs:string"/>
   <xs:element name="body" type="xs:string"/>
  </xs:sequence>
 </xs:complexType>
</xs:element>
</xs:schema>
```

# Schéma XML: association entre un document XML et XSD

**Cas sans namespace:**

- Au niveau du xml

```
<?xml version="1.0" encoding="UTF-8"?>
<personnes xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xsi:noNamespaceSchemaLocation="personnes.xsd">
    <personne>
        <nom>Ali</nom>
    </personne>
</personnes>
```

- Au niveau du xsd:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           elementFormDefault="unqualified">
  <xs:element name="personnes">
      ......
```

xs ou xsd

URI du namespace officiel de XML Schema.

- Facultatif (par défaut),
- Les éléments enfants **ne sont pas dans le namespace**, seuls les éléments racines le sont.

# Schéma XML: association entre un document XML et XSD

**Cas avec namespace:**

- Au niveau du xml

namespace

```
<?xml version="1.0" encoding="UTF-8"?>
<ns:personnes xmlns:ns="http://www.exemple.com/personne"
              xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
              xsi:schemaLocation="http://www.exemple.com/personne personnes.xsd">

    <ns:personne>
        ........
```

Pour valider tout ce qui est dans le namespace, utilise….

- Au niveau du xsd:

Les éléments du XML appartiennent à ce namespace.

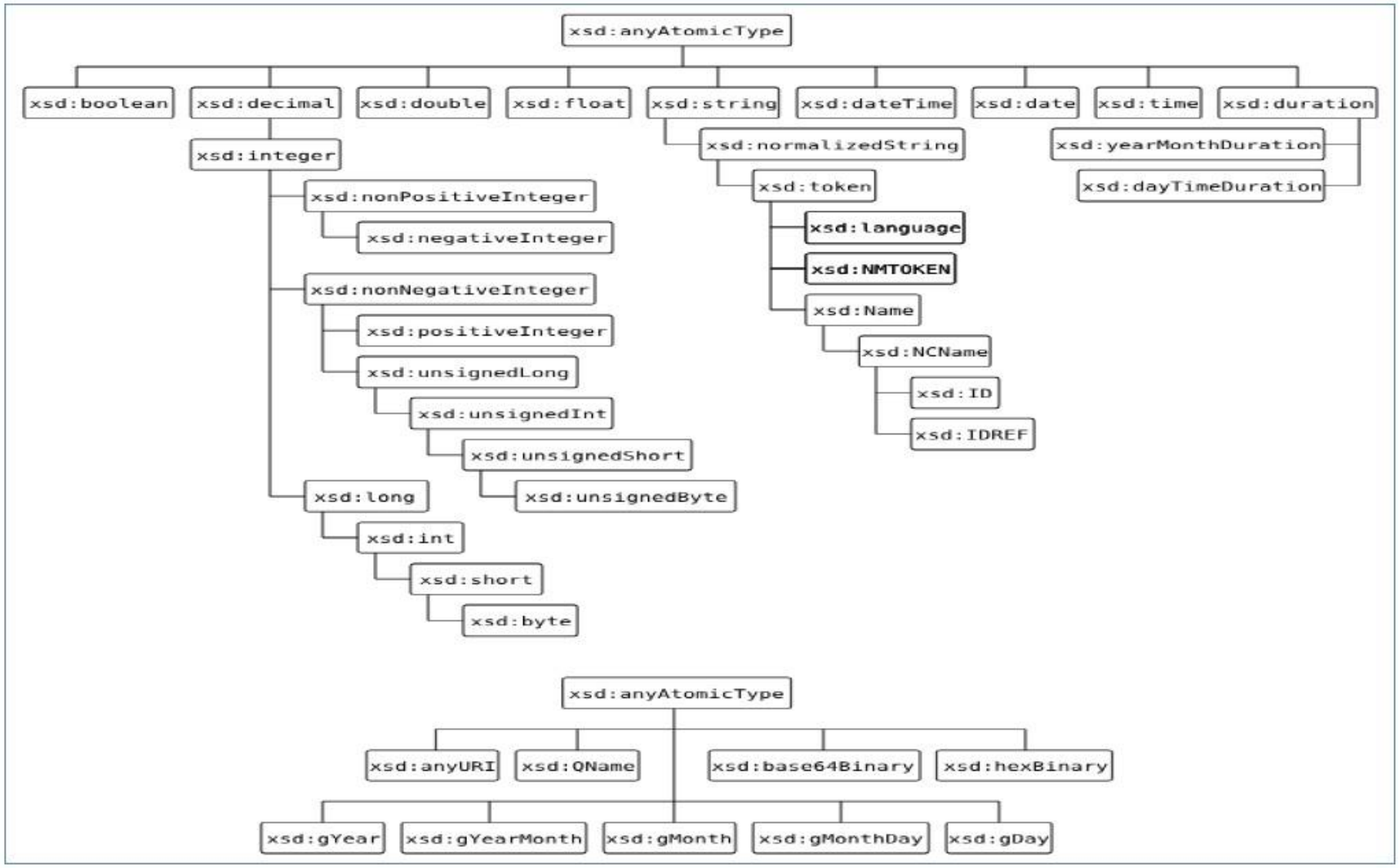
```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
           targetNamespace="http://www.exemple.com/personne"
           elementFormDefault="qualified"
           xmlns:ns="http://www.exemple.com/personne">
    .....
```

- Tous les éléments définis dans le XSD **sont associés au namespace**.
- Tous les éléments enfants (ici personne, nom, age) appartiennent au namespace.

# Schéma XML: commandes de base

- **Element**: association d'un type à une balise
  - name, type, ref, minOccurs , maxOccurs ,
- **Attribute** : association d'un type à un attribut
  - name , type
- **Type simple** : les multiples types de base
  - entier, réel, string, time, date, ID, IDREF, …,
  - extensibles par des contraintes
- **Type complexe** : une composition de types
  - définit une agrégation d'éléments typés

# Schéma XML: Types prédéfinis



# Schéma XML: Principaux types simples

- string
- byte :-128 et 127
- hexBinary : 0FB7
- integer :-126789, -1, 0, 1, 126789
- int : -2 147 483 648 et 2 147 483 647
- long : -9 223 372 036 854 775 808 et 9 223 372 036 854 775 807.
- short :-32 768 et 32 767
- float : 3.4x10^-38 et 3.4x10^38
- double : 1.7x10^-308 et 1.7x10^308
- boolean : true, false 1, 0
- anyURI : http://www.xml.com

- time : 13:20:00
- dateTime : 2020-05-31T13:20:00
- date : 2023-05-31
- gMonth : --05--
- gYear : 2030
- gYearMonth :2026-02
- gDay :---31
- gMonthDay :--05-31
- language :en-GB, en-US, fr
- ID :"A212"
- IDREF: "A212"
- IDREFS :"A212" "B213"

# Schéma XML: Principaux types simples

## float

```
<xsd:element name="nombre" type="xsd:float" />
```

```
<nombre>42</nombre>
<nombre>-42.25</nombre>
<nombre>3E4</nombre>
<nombre>10e-5</nombre>
```

## anyURI

```
<xsd:attribute name="adresse" type="xsd:anyURI"/>
```

```
<!-- URI absolu -->
<image adresse="http://www.xml.com/emplois/emploi2030.pdf" />
<!-- URI relatif -->
<image adresse="../emplois/emploi2030.pdf"/>
```

# Schéma XML: Principaux types simples

## ID et IDREF

```
<customer name="name 1" number="CUST12" type="VIP"/>
<orders count="2">
<order number="ORD100" owner="CUST12" total="900"/>
<order number="ORD101" owner="CUST12" total="200"/>
</orders>
```

```
<xs:element name="order">
<xs:complexType>
<xs:sequence>
<xs:attribute name="number" type="xs:ID" />
<xs:attribute name="owner" type="xs:IDREF" />
<xs:attribute name="total" type="xs:decimal" />
</xs:sequence>
</xs:complexType>
</xs:element>
```

# Schéma XML: élément simple

- Un élément simple est un élément XML qui peut contenir du texte seulement. Il **ne peut pas contenir** d'autres éléments ou d'attributs.

```
<?xml version="1.0" encoding="UTF-8"?>
<age>25</age>
```

**Exemple 1 : sans restriction**

```
<?xml version="1.0"?>
<xsd:schema xmlns:xsd="http://www.w3.org/2001//XMLSchema">
<xsd: element name="age" type="xsd:int" />
</xsd:schema>
```

# Schéma XML: élément simple

- Un élément simple est un élément XML qui peut contenir du texte seulement. Il **ne peut pas contenir** d'autres éléments ou d'attributs.

```
<?xml version="1.0" encoding="UTF-8"?>
<age>25</age>
```

**Exemple 2 : extensible par des contraintes (l'âge doit être entre 15 et 29)**

```
<?xml version="1.0"?>
<xs:schema xmlns:xs="http://www.w3.org/2001//XMLSchema">
<xs: element name="age">
 <xs:simpleType>
    <xs:restriction base="xs:int">
       <xs:minInclusive value="15"/>
       <xs:maxInclusive value="29"/>
    </xs:restriction>
 </xs:simpleType>
</xs:element>
```

# Schéma XML: élément simple

**1ère solution**

```xml
<?xml version="1.0" encoding="UTF-8"?>
<voiture>Audi</voiture>
<voiture>Golf</voiture>
<voiture>BMW</voiture>
```

```xml
<xs:element name="voiture">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:enumeration value="Audi"/>
   <xs:enumeration value="Golf"/>
   <xs:enumeration value="BMW"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

**2ème solution**

```xml
<xs:element name ="voiture" type="Typevoiture"/>
<xs:simpleType name="Typevoiture">
 <xs:restriction base="xs:string">
  <xs:enumeration value="Audi"/>
  <xs:enumeration value="Golf"/>
  <xs:enumeration value="BMW"/>
 </xs:restriction>
</xs:simpleType>
```

```xml
<voiture>Mercedes</voiture> <!-- ✗ pas dans l'énumération -->
```

# Schéma XML: élément simple

## Valeur par défaut et fixe pour les éléments

**Valeur par défaut :**

```
<xs:element name = "grade" type = "xs:string" default = "Pr" />
```

**Valeur fixe :**

```
<xs:element name = "devise" type = "xs:string" fixed = "DT" />
```

```
<?xml version="1.0" encoding="UTF-8"?>
<grade/>
```

ou

```
<grade>Pr</grade>
```

→Si on ne fournit pas de valeur pour grade, la valeur par défaut Pr est utilisée.

→ Pour un élément à **valeur fixe**, le XML **doit obligatoirement** contenir cette valeur

```
<?xml version="1.0" encoding="UTF-8"?>
<devise>DT</devise>
```

# Schéma XML: attribut

- Tous les attributs sont déclarés comme des types simples.
- Si un élément a des attributs, il est considéré comme étant de type complexe. Mais l'attribut lui-même est toujours déclaré comme un type simple.

**Exemple**

Voici un élément XML avec un attribut:

```
<lastname lang="EN">Smith</lastname>
```

Et voici la définition de l'attribut correspondant:

```
<xs:attribute name="lang" type="xs:string"/>
```

## Valeur par défaut et fixe pour les attributs

**Valeur par défaut :**

```
<xs:attribut name = "grade" type = "xs:string" default = "Pr" />
```

**Valeur fixe :**

```
<xs:attribut name = "devise" type = "xs:string" fixed = "DT" />
```

**Restriction :**

```
 <xs:attribut name = "code" type = "xs:int" use = "required" />
```

# Schéma XML: restriction

- Les restrictions sont utilisées pour définir des valeurs acceptables pour les éléments ou les attributs XML.
- Les restrictions sur les éléments XML sont appelées facettes.

**Restrictions sur les valeurs**

```
<xs:element name="age">
 <xs:simpleType>
  <xs:restriction base="xs:integer">
   <xs:minInclusive value="0"/>
   <xs:maxInclusive value="120"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

**Restrictions sur un ensemble de valeurs**

```
<xs:element name="car">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:enumeration value="Audi"/>
   <xs:enumeration value="Golf"/>
   <xs:enumeration value="BMW"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

# Schéma XML: restriction

- Les restrictions sont utilisées pour définir des valeurs acceptables pour les éléments ou les attributs XML.
- Les restrictions sur les éléments XML sont appelées facettes.

**Restrictions sur une série de valeurs**

```
<xs:element name="letter">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:pattern value="[a-z]"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

**Expressions possibles**

- `<xs:pattern value="[A-Z][A-Z][A-Z]"/>`
- `<xs:pattern value="[a-zA-Z][a-zA-Z][a-zA-Z]"/>`
- `<xs:pattern value="[xyz]"/>`
- `<xs:pattern value="[0-9][0-9][0-9][0-9][0-9]"/>`
- `<xs:pattern value="([a-z])*"/>`
- `<xs:pattern value="([a-z][A-Z])+"/>`
- `<xs:pattern value="m|f"/>`

# Schéma XML: restriction

- Les restrictions sont utilisées pour définir des valeurs acceptables pour les éléments ou les attributs XML.
- Les restrictions sur les éléments XML sont appelées facettes.

- Restriction sur la longueur

```
<xs:element name="password">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:length value="8"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

```
<xs:element name="password">
 <xs:simpleType>
  <xs:restriction base="xs:string">
   <xs:minLength value="5"/>
   <xs:maxLength value="8"/>
  </xs:restriction>
 </xs:simpleType>
</xs:element>
```

# Schéma XML: élément complexe

- Un élément complexe est un élément XML qui contient d'autres éléments et / ou des attributs. Il existe quatre types d'éléments complexes:

  - Eléments vides

  - Contenu standard : il est composé d'éléments ou d'attributs
    - sequence : collection ordonnée d'éléments typés
    - all : collection non ordonnée d'éléments typés
    - choice : choix entre éléments typés

  - Contenu simple : il est composé d'attributs et d'un texte de type simple <xsd:simpleContent>

  - Contenu mixte : il est composé d'attributs, d'éléments et de texte

# Schéma XML: élément complexe

- **Les éléments vides:**

**Définition** 1:seul l'élément product peut utiliser le type complexe spécifié

```
<!-- Définition de l'élément racine -->
  <xs:element name="product">
    <xs:complexType>
      <xs:attribute name="prodid"
type="xs:integer"/>
    </xs:complexType>
  </xs:element>
```

```
<product prodid="1345" />
```

**Définition** 2: plusieurs éléments peuvent se référer au même type complexe:

```
 <!-- Déclaration de l'élément racine
et association au type -->
  <xs:element name="product"
type="ProductType"/>
  <!-- Définition du type utilisé par
product -->
  <xs:complexType name="ProductType">
    <xs:attribute name="prodid"
type="xs:integer" />
  </xs:complexType>
```

# Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent des attributs :

```
<personne nom= "Mabrouk" prenom="Mouna"/>
```

```
<personne nom="Mabrouk"/>
```

```
<xsd:element name="personne">
   <xsd:complexType>
     <xsd:attribute name="nom" type="xsd:string" use="required"/>
     <xsd:attribute name="prenom" type="xsd:string"/>
   </xsd:complexType>
 </xsd:element>
```

Obligatoire

Optionnelle

# Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent d'autres éléments:
  - Sequence

```
<personne>
  <nom>Mabrouk</nom>
  <prenom>Mouna</prenom>
</personne>
```

```
<xsd:element name="personne">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="prenom" type="xsd:string"/>
      </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

- Ordre **obligatoire** : les éléments doivent apparaître dans l'ordre défini.
- Multiplicité contrôlé (avec minOccurs, maxOccurs).

# Schéma XML: élément complexe

- **Contenu standard:** Indicateurs d'occurrence : Il est possible de contrôler le nombre minimal et maximal d'un élément par l'utilisation de **minOccurs** et **maxOccurs**

```
<personne>
  <nom>Mabrouk</nom>
  <prenom>Mouna</prenom>
  <email>mouna.Mabrouk@example.com</email>
  <email>contact.mouna@example.com</email>
</personne>
```

```
<xsd:element name="personne">
    <xsd:complexType>
      <xsd:sequence>
        <xsd:element name="nom" type="xsd:string"/>
        <xsd:element name="prenom" type="xsd:string"/>
        <xsd:element name="email" type="xsd:string" minOccurs="1"
maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

un nombre illimité de fois

# Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent d'autres éléments:
  - all

```
<personne>
  <prenom>Mouna</prenom>
  <email>mouna.mabrouk@example.com</email>
  <nom>mabrouk</nom>
</personne>
```

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:all>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
      <xsd:element name="email" type="xsd:string" minOccurs="1"/>
    </xsd:all>
  </xsd:complexType>
</xsd:element>
```

- **Tous** les éléments doivent apparaître une fois (ou zéro fois si minOccurs="0"), mais **l'ordre n'a pas d'importance**.
- Chaque élément peut apparaître **au maximum** une fois (maxOccurs doit être 1)

# Schéma XML: élément complexe

- **Contenu standard:** Les éléments qui contiennent d'autres éléments:
  - Choice

```
<personne>
  <nom>Mabrouk</nom>
</personne>
```

```
<xsd:element name="personne">
  <xsd:complexType>
    <xsd:choice>
      <xsd:element name="nom" type="xsd:string"/>
      <xsd:element name="prenom" type="xsd:string"/>
      <xsd:element name="email" type="xsd:string"/>
    </xsd:choice>
  </xsd:complexType>
</xsd:element>
```

- L'élément peut contenir un **seul** des sous éléments définis.
- Permet des alternatives.

# Schéma XML: élément complexe

- **Contenu simple:** Les éléments qui contiennent du texte et des attributs:

```
<ville pays="Tunisie" population="200000">Tunis</ville>
```

```
<xsd:element name="ville">
  <xsd:complexType>
    <xsd:simpleContent>
      <xsd:extension base="xsd:string">
        <xsd:attribute name="pays" type="xsd:string" use="required"/>
        <xsd:attribute name="population" type="xsd:integer"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:element>
```

# Schéma XML: élément complexe

- **Contenu mixte:** Les éléments qui contiennent du texte et d'autres éléments :

```
<paragraphe>
  Bonjour, ceci est un
  <mot>exemple</mot>
  de contenu mixte
  <mot>XML</mot>.
</paragraphe>
```

```
<xsd:element name="paragraphe">
    <xsd:complexType mixed="true">
      <xsd:sequence>
        <xsd:element name="mot" type="xsd:string" minOccurs="0"
maxOccurs="unbounded"/>
      </xsd:sequence>
    </xsd:complexType>
</xsd:element>
```

# Schéma XML: élément complexe

- **Contenu mixte:** Les éléments qui contiennent du texte, d'autres éléments et des attributs

```
<paragraphe langue="fr" id="1">
  Bonjour, ceci est un
  <mot>exemple</mot>
  de contenu mixte
  <mot>XML</mot>.
</paragraphe>
```

```
<xsd:element name="paragraphe">
  <xsd:complexType mixed="true">
    <xsd:sequence>
      <xsd:element name="mot" type="xsd:string" minOccurs="0"
maxOccurs="unbounded"/>
    </xsd:sequence>
    <xsd:attribute name="langue" type="xsd:string"/>
    <xsd:attribute name="id" type="xsd:integer"/>
  </xsd:complexType>
</xsd:element>
```

# Schéma XML: élément complexe

- **Réutilisation des éléments**: On déclare de manière globale les différents éléments qui composent notre Schéma XML, puis, dans nos structures complexes, on y fait référence par le mot clef « **ref** ».

```
<xs:element name="note">
    <xs:complexType>
      <xs:sequence>
        <xs:element ref="to"/>
        <xs:element ref="from"/>
        <xs:element ref="heading"/>
        <xs:element ref="body"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <xs:element name="to" type="xs:string"/>
  <xs:element name="from" type="xs:string"/>
  <xs:element name="heading" type="xs:string"/>
  <xs:element name="body" type="xs:string"/>
```

```
<note>
  <to>Alice</to>
  <from>Bob</from>
  <heading>Rappel</heading>
  <body>N'oublie pas notre
réunion demain à 10h.</body>
</note>
```

# Schéma XML: élément complexe

- **Exercice :** Donner un schéma xml pour le fichier " shiporder.xml " suivant:

```
?xml version="1.0" encoding="ISO-8859-1"?>
<shiporder orderid="889923">
  <orderperson>John Smith</orderperson>
  <shipto>
    <name>Ola Nordmann</name>
    <address>Langgt 23</address>
    <city>4000 Stavanger</city>
    <country>Norway</country>
  </shipto>
  <item>
    <title>Empire Burlesque</title>
    <note>Special Edition</note>
    <quantity>1</quantity>
    <price>10.90</price>
  </item>
  <item>
    <title>Hide your heart</title>
    <quantity>1</quantity>
    <price>9.90</price>
  </item>
</shiporder>
```

# Schéma XML: élément complexe

- **Solution I:**

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="shiporder">
    <xs:complexType>
      <xs:sequence>

        <xs:element name="orderperson" type="xs:string"/>

        <xs:element name="shipto">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="name" type="xs:string"/>
              <xs:element name="address" type="xs:string"/>
              <xs:element name="city" type="xs:string"/>
              <xs:element name="country" type="xs:string"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>
```

# Schéma XML: élément complexe

- **Solution I :**

```
<xs:element name="item" maxOccurs="unbounded">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="title" type="xs:string"/>
              <xs:element name="note" type="xs:string" minOccurs="0"/>
              <xs:element name="quantity" type="xs:positiveInteger"/>
              <xs:element name="price" type="xs:decimal"/>
            </xs:sequence>
          </xs:complexType>
        </xs:element>

      </xs:sequence>
      <xs:attribute name="orderid" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>

</xs:schema>
```

# Schéma XML: élément complexe

- **Solution 2** : La méthode précédente est très simple, mais peut être difficile à lire et à maintenir lorsque les documents sont complexes.
  La méthode de conception suivante est basée sur la définition préalable de tous les éléments et attributs, puis sur leur référence à l'aide de **l'attribut ref**

```
<?xml version="1.0" encoding="UTF-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- definition of simple elements -->
  <xs:element name="orderperson" type="xs:string"/>
  <xs:element name="name" type="xs:string"/>
  <xs:element name="address" type="xs:string"/>
  <xs:element name="city" type="xs:string"/>
  <xs:element name="country" type="xs:string"/>
  <xs:element name="title" type="xs:string"/>
  <xs:element name="note" type="xs:string"/>
  <xs:element name="quantity" type="xs:positiveInteger"/>
  <xs:element name="price" type="xs:decimal"/>
```

# Schéma XML: élément complexe

- **Solution 2** :

```
<!-- definition of attributes -->
<xs:attribute name="orderid" type="xs:string"/>

<!-- definition of complex elements -->
<xs:element name="shipto">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="name"/>
      <xs:element ref="address"/>
      <xs:element ref="city"/>
      <xs:element ref="country"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

# Schéma XML: élément complexe

- **Solution 2** :

```
<xs:element name="item">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="title"/>
      <xs:element ref="note" minOccurs="0"/>
      <xs:element ref="quantity"/>
      <xs:element ref="price"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:element name="shiporder">
  <xs:complexType>
    <xs:sequence>
      <xs:element ref="orderperson"/>
      <xs:element ref="shipto"/>
      <xs:element ref="item" maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute ref="orderid" use="required"/>
  </xs:complexType>
</xs:element>
</xs:schema>
```

# XSL

- XSL: Le langage XSL (eXtensible Stylesheet Language), permet d'effectuer des transformations sur un document XML.

- Conçu pour transformer des documents XML en d'autres formats comme PDF ou des pages HTML.

- Il est constitué de 2 langages:
  - **XSLT (pour XML Stylesheet Language Transformation)** est un langage de transformation de documents XML permettant d'appliquer des règles de transformation sur un document XML donné en entrée, et de générer en sortie un document XML, HTML ou une autre forme.

  - **XSL-FO (pour XML Stylesheet Language - Formatting Objets)** est un langage de mise en page de document.

# XSL: principe de fonctionnement

- Une transformation XSLT- appelée une feuille de style – consiste en une série de **règles** pour transformer **un arbre source** (source tree) XML en un arbre résultant (result tree).

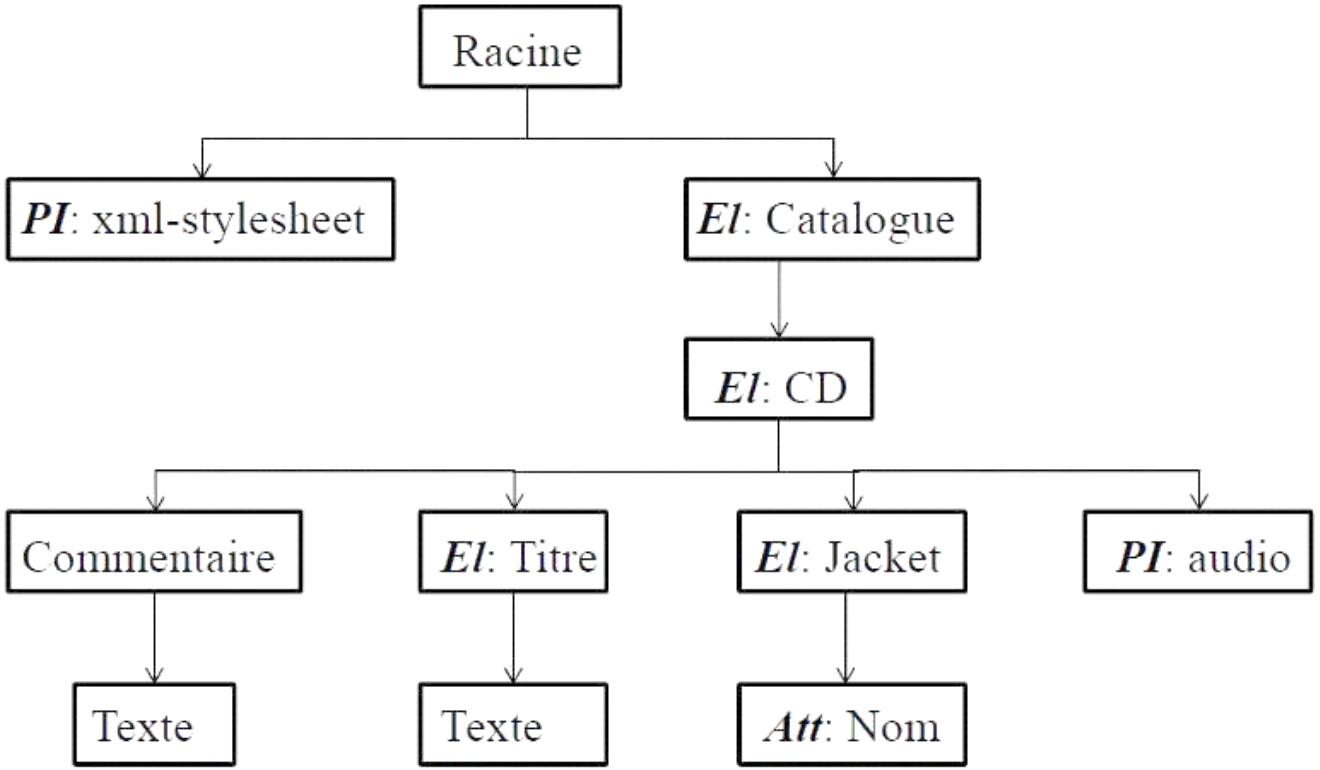
- La transformation se fait en associant des **motifs** (patterns) à des **modèles** (templates)et en les appliquant aux éléments de l'arbre source.

- Exemple: catalogue.xml

```
<?xml version="1.0" standalone="yes"?>
<?xml-stylesheet type="text/xsl" href="transfo.xsl"?>
<Catalogue>
  <CD>
      <!-- Bon CD -->
      <Titre> Best of Bob Dylan<Titre>
      <Jacket NOM="dylan.gif"/ >
      <?audio dylan.mp3?>
  </CD>
</Catalogue>
```

# XSL: principe de fonctionnement

- L'arbre associé:



# XSL: structure d'une feuille XSLT

- Toutes les instructions XSL appartiennent à l'espace de noms xsl, elles commencent toutes par xsl:

```
<?xml version="1.0" ?>
<xsl:stylesheet
    xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
    version="1.0">
    <!-- les règles de transformations -->
    .......
</xsl:stylesheet>
```

# XSL: format d'une règle de transformation

- Un **motif** est une expression qui identifie le/les noeud(s) XML du document qui est/sont concerné(s) par la règle et sur le(s) quel(s) il faut appliquer une action

- Une **action** est une opération qui réalise la transformation et/ou spécifie les caractéristiques de la présentation

```
<xsl:template match='un motif'>
    [action]
</xsl:template>
```

- Exemple demo.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="demo.xsl"?>
<demoXML>
    <message>
        Voici du XML
    </message>
</demoXML>
```

# XSL: format d'une règle de transformation

- Exemple demo.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="demo.xsl"?>
<demoXML>
    <message>
        Voici du XML
    </message>
</demoXML>
```

- Demo.xsl

```
<?xml version="1.0"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:template match="/">
    <html>
        <body>
            <xsl:value-of select="demoXML/message"/>
        </body>
    </html>
</xsl:template>
</xsl:stylesheet>
```

- Résultat:

```
<!DOCTYPE html>
<html>
   <body>
        Voici du XML
    </body>
</html>
```

# XSL: format d'une règle de transformation

- Exemple demo.xml

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="demo.xsl"?>
<demoXML>
    <message>
        Voici du XML
    </message>
</demoXML>
```

- Demo.xsl

```
<?xml version="1.0"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
<xsl:template match="/">
    <html>
        <body>
            <xsl:value-of select="demoXML/message"/>
        </body>
    </html>
</xsl:template>
</xsl:stylesheet>
```

mentionne les éléments qui sont concernés par la transformation, / indique l'élément racine et tous les sous éléments

permet de sélectionner un élément du fichier XML pour le traiter dans le fichier XSL. Dans l'attribut select, on détermine le chemin d'accès vers la balise XML souhaitée.

- Résultat:

```
<!DOCTYPE html>
<html>
   <body>
       Voici du XML
    </body>
</html>
```

# XSL: format d'une règle de transformation

- Exemple enfant.xml

```xml
<enfants>
    <enfant>
        <nom>Ali</nom>
        <lien>garçon</lien>
        <date>07/11/83</date>
        <data>Une tête brulée!</data>
    </enfant>
    <enfant>
        <nom>Samia</nom>
        <lien>fille</lien>
        <date>20/12/85</date>
        <data>La petite fille chérie à son papa.</data>
    </enfant>
</enfants>
```

# XSL: format d'une règle de transformation

- Exemple enfant.xsl

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0
  <!-- Template principal -->
  <xsl:template match="/">
    <html>
      <head>
        <title>Liste des enfants</title>
      </head>
      <body>
        <h2>Liste des enfants</h2>
        <table border="1">
          <tr>
            <th>Nom</th>
            <th>Lien</th>
            <th>Date</th>
            <th>Remarques</th>
          </tr>
          <!-- Boucle sur chaque enfant -->
          <xsl:for-each select="enfants/enfant">
            <tr>
              <td><xsl:value-of select="nom"/></td>
              <td><xsl:value-of select="lien"/></td>
              <td><xsl:value-of select="date"/></td>
              <td><xsl:value-of select="data"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>
```

Pour afficher tous les éléments, on ajoute la balise: xsl:for-each (pour chaque) avec comme attribut select="enfants/enfant"

# XSL: format d'une règle de transformation

- Résultat

```
<!DOCTYPE html>
<html>
   <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>Liste des enfants</title>
   </head>
   <body>
      <h2>Liste des enfants</h2>
      <table border="1">
         <tr>
            <th>Nom</th>
            <th>Lien</th>
            <th>Date</th>
            <th>Remarques</th>
         </tr>
         <tr>
            <td>Ali</td>
            <td>garçon</td>
            <td>07/11/83</td>
            <td>Une tête brulée!</td>
         </tr>
         <tr>
            <td>Samia</td>
            <td>fille</td>
            <td>20/12/85</td>
            <td>La petite fille chérie à son papa.</td>
         </tr>
      </table>
   </body>
</html>
```

Liste des enfants

| Nom | Lien | Date | Remarques |
|---|---|---|---|
| Ali | garçon | 07/11/83 | Une tête brulée! |
| Samia | fille | 20/12/85 | La petite fille chérie à son papa. |

# XSL: format d'une règle de transformation

- Le langage XSLT permet aussi de :
  - trier les données XML en ordre croissant ou décroissant.
  - filtrer des éléments XML en fonction de certains critères.
  - choisir des éléments.
  - retenir des éléments par des tests conditionnels.

# XPath

- XSLT s'appuie sur XPath pour désigner une partie d'un arbre XML
- XPath est le langage de description des nœuds dans XSLT.
- Il est aussi utilisé comme langage de requêtes dans les bases de données XML.
- XPath est un mécanisme (syntaxe+fonctions) permettant d'extraire des informations d'un document XML.

```
<messages>
    <message numero="1" date="2025-10-01">
        <dest>promo2025</dest>
        <dest bcc="oui">Mouna Jarraya</dest>
        <contenu>Bon courage !</contenu>
    </message>

    <message numero="2" date="2025-10-01">
...
```

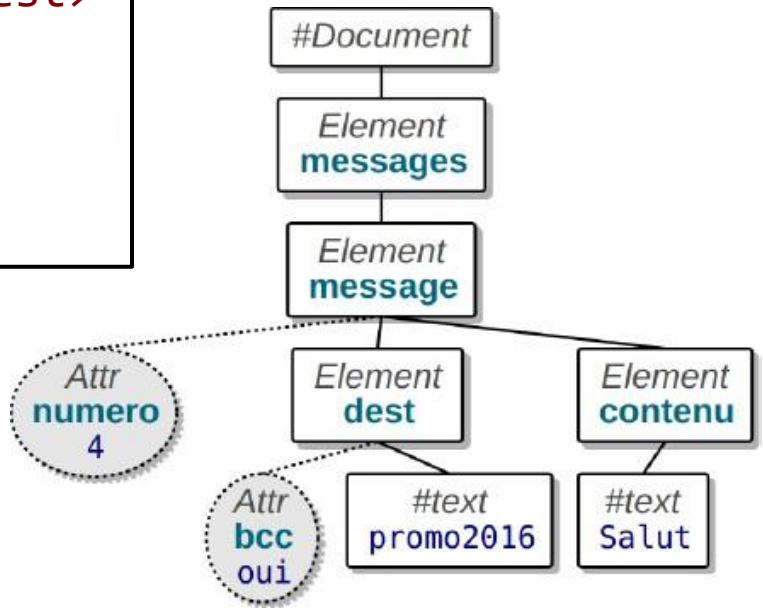
Extraire le contenu du message n°4 s'écrit ainsi en XPath :

```
/messages/message[@numero=4]/contenu
```

# XPath

- XPath sert à extraire des informations dans un arbre XML à l'aide d'une sorte de chemin.

```
<?xml version="1.0"?>
<messages>
……
    <message numero="4">
        <dest bcc="oui">promo2016</dest>
        <contenu>Salut</contenu>
    </message>
……
</messages>
```

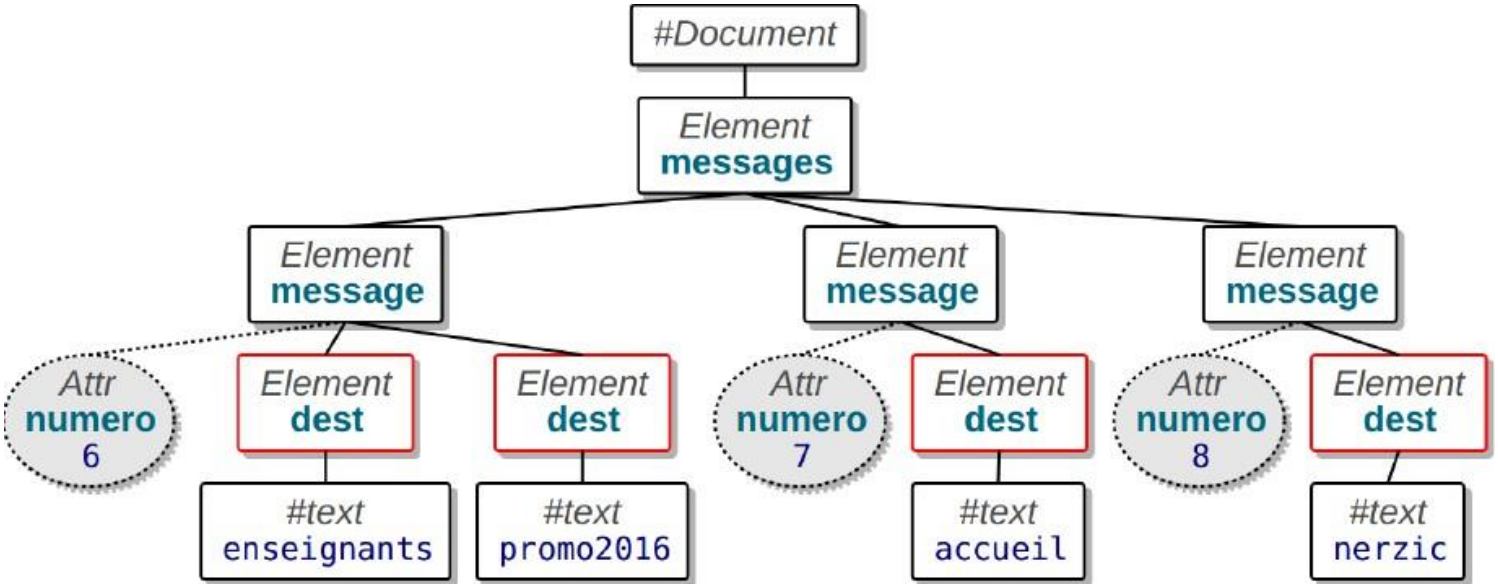



- Le but d'XPath est d'aller chercher les informations voulues dans le document XML. Ex: quels sont les destinataires du message n°4 ?

```
/messages/message[@numero="4"]/dest
```

# XPath

- Une expression XPath peut retourner plusieurs réponses.
- En effet, un élément parent peut contenir plusieurs exemplaires du même élément enfant.
- Le chemin /messages/message/dest sélectionne 4 noeuds (rouge).



# XPath

- Une expression XPath est une suite d'étapes séparées par des séparateurs :
  [sep] étape1 sep étape2 sep étape3…

- Les étapes sont les noms des éléments dans lesquels il faut aller successivement.

- Cependant cela dépend du séparateur employé :

- / Ce séparateur se comporte comme dans Unix : s'il est mis au début du chemin, il représente le document entier ; s'il est mis entre les étapes, c'est un simple séparateur.

- // Ce séparateur signifie de sauter un nombre quelconque d'éléments quelconques. C'est un peu comme si on écrivait /*/*/*.../ un nombre indéfini de fois, y compris 0. S'il est au début du chemin, cela signifie alors que la première étape est à chercher n'importe où dans l'arbre.

# XPath

- Voici quelques exemples de chemins :
  - /messages/message/dest sélectionne tous les éléments <dest> de tous les éléments <message> de la racine <messages>.
  - //dest sélectionne tous les éléments <dest> où qu'ils soient dans l'arbre. Ça donne le même résultat que l'exemple précédent parce qu'il n'y a pas de <dest> ailleurs.
  - /messages//contenu sélectionne tous les éléments <contenu> situés sous la racine <messages>.
- Pour désigner un attribut et non pas un sous-élément, on met un @ devant le nom de l'attribut:
  - /messages/message/@numero sélectionne tous les nœuds attributs nommés numero des éléments <message>.
  - //@numero sélectionne tous les nœuds attributs portant ce nom n'importe où dans l'arbre.

# XQuery

- XQuery est le langage pour interroger les données XML.
- XQuery pour XML est comme SQL pour les bases de données.
- XQuery est construit sur les expressions XPath.
- XQuery est une recommandation du W3C.

# XQuery

- XQuery peut être utilisé pour :
  - Extraire des informations à utiliser dans un service Web
  - Générer des rapports de synthèse
  - Transformer des données XML en XHTML
  - Rechercher des documents Web pour obtenir des informations pertinentes

# XQuery

- Exemple : books.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<bookstore>

<book category="COOKING">
  <title lang="en">Everyday Italian</title>
  <author>Giada De Laurentiis</author>
  <year>2005</year>
  <price>30.00</price>
</book>

<book category="CHILDREN">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
....
```

- XQuery utilise des fonctions pour extraire des données des documents XML.
- La fonction doc() est utilisée pour ouvrir le fichier "books.xml".

```
doc("books.xml")
```

- XQuery utilise des expressions de chemin (path expressions) pour naviguer à travers les éléments d'un document XML.
- L'expression de chemin suivante est utilisée pour sélectionner tous les éléments <title> dans le fichier "books.xml".

```
doc("books.xml")/bookstore/book/title
```

Résultat :

```
<title lang="en">Everyday Italian</title>
<title lang="en">Harry Potter</title>
<title lang="en">XQuery Kick Start</title>
<title lang="en">Learning XML</title>
```

# XQuery

- Exemple : books.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<bookstore>

<book category="COOKING">
  <title lang="en">Everyday Italian</title>
  <author>Giada De Laurentiis</author>
  <year>2005</year>
  <price>30.00</price>
</book>

<book category="CHILDREN">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
....
```

- XQuery utilise des prédicats pour limiter les données extraites des documents XML.
- Le prédicat suivant est utilisé pour sélectionner tous les éléments <book> sous l'élément <bookstore> qui ont un élément <price> dont la valeur est inférieure à 30 :

```
doc("books.xml")/bookstore/book[price<30]
```

Résultat :

```
<book category="CHILDREN">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
```

# XQuery

- **FLWOR** (prononcé « flower ») est un acronyme pour « For, Let, Where, Order by, Return »:
  - **For** sélectionne une séquence de nœuds
  - **Let** assigne une séquence à une variable
  - **Where** filtre les nœuds
  - **Order by** trie les nœuds
  - **Return** définit ce qui doit être renvoyé (évalué une fois pour chaque nœud)

# XQuery

```
doc("books.xml")/bookstore/book[price>30]/title
```

- L'expression ci-dessus sélectionnera tous les éléments <title> sous les éléments <book> qui sont eux-mêmes sous l'élément <bookstore> et qui ont un élément <price> dont la valeur est supérieure à 30.

- L'expression FLWOR suivante sélectionnera exactement la même chose que l'expression de chemin ci-dessus :

```
for $x in doc("books.xml")/bookstore/book
where $x/price>30
return $x/title
```

- Résultat :

```
<title lang="en">XQuery Kick Start</title>
<title lang="en">Learning XML</title>
```

# XQuery

- Avec FLWOR, vous pouvez trier le résultat :

```
for $x in doc("books.xml")/bookstore/book
where $x/price>30
order by $x/title
return $x/title
```

- Résultat :

```
<title lang="en">Learning XML</title>
<title lang="en">XQuery Kick Start</title>
```

- La clause for sélectionne tous les éléments <book> sous l'élément <bookstore> et les place dans une variable appelée $x.
- La clause where sélectionne uniquement les éléments <book> dont l'élément <price> a une valeur supérieure à 30.
- La clause order by définit l'ordre de tri. Ici, le tri se fait sur l'élément <title>.
- La clause return spécifie ce qui doit être renvoyé. Ici, elle renvoie les éléments <title>.

# Comparaison XML vs CSV

- CSV est uniquement adapté pour des cas très simples i.e. quand les conditions suivantes sont réunies :
  - Données tabulaires
  - Au plus un élément par case
  - Pas de niveau hierarchique entre les éléments
- Dans tous les autres cas, XML est un meilleur choix.

# Comparaison XML vs JSON

- JSON se base sur 2 structures simples :
  - {...} : les accolades qui définissent une collection de couples clé:valeur séparés par des virgules et où la clé est une chaîne de caractères.
  - [...] : Les crochets qui définissent un tableau de valeurs séparés par des virgules.

→Une valeur est soit une collection, un tableau ou un type simple (chaîne, entier, etc...)

| XML | JSON |
|---|---|
| <employees><br><employee><br><firstName>John</firstName><br><lastName>Doe</lastName><br></employee><br><employee><br><firstName>Anna</firstName><br><lastName>Smith</lastName><br></employee><br><employee><br><firstName>Peter</firstName><br><lastName>Jones</lastName><br></employee><br></employees> | {"employees":[<br>{ "firstName":"John",<br>"lastName":"Doe" },<br>{ "firstName":"Anna",<br>"lastName":"Smith" },<br>{ "firstName":"Peter",<br>"lastName":"Jones" }<br>]} |

# Comparaison XML vs JSON

- Avantages du JSON :
  - présente une structure hiérarchique comme le XML
  - prends moins de place.

- Avantages du XML :
  - Vérifier la validité d'un document : lors d'un problème on peut s'assurer facilement que ce dernier ne provient pas de la structure du document via les Schémas XML/Doctypes. Ces derniers jouent le rôle de contrat à respecter entre les différentes applications manipulant les données.

  - Attacher des métadonnées via les attributs : en JSON, nous n'avons pas la possibilité de distinguer les données des métadonnées.

  - Le très grand nombre de langages, standards, et applications développés autour du XML : xpath, xslt, SOAP, Xquery,… Ces outils de traitement simplifie considérablement la réalisation et la manipulation des données structurées via le XML.